

SOP-SH-05 — Indexing Strategy

Kode	SOP-SH-05
Pemilik	Shikamaru (DBA / Data Architect)
Revisi	1.0 · Berlaku 2026-05-29 · Reviu per kuartal
Referensi	Postgres index types, DAMA-DMBOK (Data Operations), tools/query-optimization-framework.md
COBIT	APO14 (Managed Data), DSS06 (Business Process Controls)

1. Tujuan

Memastikan index ditambahkan **hanya berdasarkan pola query terbukti** (EXPLAIN), tepat tipe (btree/gin/gist/brin/partial/covering/composite), **tanpa redundansi**, dan write-penalty terkendali — karena **tiap index ada cost di write**.

2. Ruang Lingkup

- **Berlaku**: penambahan/penghapusan index pada tabel real DB, berdasarkan pola query proven.
- **Tidak berlaku**: desain skema awal (index plan awal masuk SOP-SH-01); eksekusi index di prod (lewat SOP-SH-02 Migration).

3. Definisi & Istilah

- **btree** — index default; untuk equality, range, ORDER BY.
- **gin** — untuk array, jsonb, full-text search.
- **gist** — untuk geometric, range type, fuzzy.
- **brin** — untuk tabel raksasa append sekuensial (time-series).
- **Partial index** — index hanya subset baris (cth `WHERE deleted_at IS NULL`).
- **Covering index** — `INCLUDE` kolom tambahan supaya gak perlu heap lookup.
- **Composite index** — gabungan kolom; **urutan penting** (hanya left-most prefix yang usable).
- **Write penalty** — tiap index harus di-update saat INSERT/UPDATE → tambah biaya tulis.

4. Referensi

Postgres index types, anti-pattern "index sembarangan" & "redundant index" (PLAYBOOK §6), kontrol D5, [query-optimization-framework](#).

5. Tanggung Jawab (RACI)

Aktivitas	R	A	C	I
Pilih & justifikasi index	Shikamaru	Shikamaru	@saitama (kalau dari query BE)	@kakashi
Deploy index di prod	@levi	Shikamaru	—	@nami

6. Prosedur

6.1 Justifikasi (gate masuk)

- 6.1.1 Buktikan **pola query** butuh index via **EXPLAIN ANALYZE** (SOP-SH-03): ada seq scan di tabel > 10rb row + WHERE/JOIN/ORDER BY yang sering?
- 6.1.2 **Exit kalau gak proven** → **jangan add** (no index sembarangan). Tabel kecil / query jarang = seq scan OK.

6.2 Pilih tipe & bentuk

- 6.2.1 **Pilih tipe** sesuai pola: equality/range/sort → btree · jsonb/array/full-text → gin · range/geo/fuzzy → gist · tabel raksasa append → brin.
- 6.2.2 **Optimasi bentuk**: filter soft-delete dominan → **partial** (`WHERE deleted_at IS NULL`); query butuh kolom ekstra → **covering** (`INCLUDE`); multi-kolom → **composite** dengan urutan = selektivitas / pola WHERE (left-most prefix).
- 6.2.3 **Cek redundansi**: index `(a, b)` sudah cover `(a)` → jangan bikin `(a)` terpisah (kontrol D5).

6.3 Validasi & deploy (exit)

- 6.3.1 Ukur **before/after** EXPLAIN ANALYZE — index benar dipakai planner & latency turun.
- 6.3.2 **Assess write penalty** — tabel write-heavy? jumlah index sudah banyak? kalau benefit baca < cost tulis → tahan.
- 6.3.3 Deploy via **SOP-SH-02** (di prod pakai `CREATE INDEX CONCURRENTLY` biar gak lock).
- 6.3.4 **Exit criteria**: index proven dipakai + no redundant + write penalty acceptable → log + handoff deploy @levi.

7. Pengecualian

- **UNIQUE/PK constraint**: index-nya wajib (bukan opsional perf) — bagian integritas, bukan SOP ini.
- **Index eksperimen**: boleh di staging buat ukur, **jangan** lolos ke prod tanpa proven.

8. Rekaman & Retensi

Rekaman	Lokasi	Retensi
Justifikasi index + EXPLAIN before/after	log.md	Permanen
Index migration (CONCURRENTLY)	repo migration	Permanen

9. Indikator Kinerja (KPI)

KPI	Rumus	Target
Index redundan/tak-berdasar	# index tanpa pattern proven / redundan	0
Index dipakai planner	# index baru yang benar dipakai ÷ total index baru	100%
Write penalty breach	# tabel write-heavy yang over-index	0

10. Riwayat Revisi

Rev	Tanggal	Perubahan
1.0	2026-05-29	Terbit pertama